Let me provide a comprehensive explanation of hashing and hash tables, building from fundamental concepts to practical implementations.

LECTURE PART

1. Hashing and Hash Tables

A hash table is a data structure that implements an associative array abstract data type, a structure that can map keys to values. The core idea is to use a hash function to transform a key into an array index, allowing for very fast insertion and lookup operations.

Let's start with a basic implementation to illustrate the concept:

```java
public class BasicHashTable<K, V> {
    private class Entry {
        K key;
        V value;

        Entry(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private Entry[] table;
    private int size;
    private static final int INITIAL_CAPACITY = 16;
    private static final double LOAD_FACTOR = 0.75;

    @SuppressWarnings("unchecked")
    public BasicHashTable() {
        // Initialize hash table with default capacity
        table = (Entry[]) new BasicHashTable.Entry[INITIAL_CAPACITY];
        size = 0;
    }
}
```

2. Hash Functions

Hash functions are the heart of hash tables. A good hash function should:
- Be deterministic (same input always produces same output)
- Distribute values uniformly across the table
- Be efficient to compute
- Minimize collisions

Here are some common hash function implementations:

```java
public class HashFunctions {
    // Division Method
```

```
    public static int divisionHash(int key, int tableSize) {
        return Math.abs(key) % tableSize;
    }

    // Multiplication Method
    public static int multiplicationHash(int key, int tableSize) {
        double A = 0.6180339887; // (√5 - 1)/2
        double temp = key * A;
        double fractional = temp - Math.floor(temp);
        return (int)(tableSize * fractional);
    }

    // String Hashing (Java's approach)
    public static int stringHash(String key) {
        int hash = 0;
        for (int i = 0; i < key.length(); i++) {
            hash = 31 * hash + key.charAt(i);
        }
        return Math.abs(hash);
    }
}
```

3. Collision Resolution

When two keys hash to the same index, we have a collision. Here are implementations of different collision resolution techniques:

```java
public class HashTable<K, V> {
    // Linear Probing Implementation
    private int linearProbe(int originalHash, int attempt) {
        return (originalHash + attempt) % table.length;
    }

    // Quadratic Probing Implementation
    private int quadraticProbe(int originalHash, int attempt) {
        return (originalHash + attempt * attempt) % table.length;
    }

    // Double Hashing Implementation
    private int doubleHash(K key, int attempt) {
        int hash1 = primaryHash(key);
        int hash2 = secondaryHash(key);
        return (hash1 + attempt * hash2) % table.length;
    }

    private int primaryHash(K key) {
        return Math.abs(key.hashCode() % table.length);
    }

    private int secondaryHash(K key) {
```

```
        // Secondary hash must be relatively prime to table size
        return 1 + Math.abs(key.hashCode() % (table.length - 1));
    }
}
```

LAB IMPLEMENTATION

Here's a complete implementation of a hash table with various features:

```java
public class AdvancedHashTable<K, V> {
    private static class Entry<K, V> {
        K key;
        V value;
        boolean isDeleted;  // For lazy deletion

        Entry(K key, V value) {
            this.key = key;
            this.value = value;
            this.isDeleted = false;
        }
    }

    private Entry<K, V>[] table;
    private int size;
    private int capacity;
    private final double loadFactor;

    @SuppressWarnings("unchecked")
    public AdvancedHashTable(int initialCapacity, double loadFactor) {
        this.capacity = initialCapacity;
        this.loadFactor = loadFactor;
        this.table = (Entry<K, V>[]) new Entry[capacity];
        this.size = 0;
    }

    public void put(K key, V value) {
        if (key == null) throw new IllegalArgumentException("Key cannot be null");

        // Check if resize is needed
        if ((double) size / capacity >= loadFactor) {
            resize();
        }

        int hash = getHash(key);
        int index = hash;
        int attempt = 1;

        while (table[index] != null && !table[index].isDeleted) {
            if (table[index].key.equals(key)) {
                // Update existing key
```

```
            table[index].value = value;
            return;
        }
        // Quadratic probing
        index = (hash + attempt * attempt) % capacity;
        attempt++;

        if (attempt >= capacity) {
            // Table is full or probing cycle completed
            resize();
            put(key, value);  // Recursively try insertion after resize
            return;
        }
    }

    table[index] = new Entry<>(key, value);
    size++;
}

public V get(K key) {
    if (key == null) return null;

    int hash = getHash(key);
    int index = hash;
    int attempt = 1;

    while (table[index] != null) {
        if (!table[index].isDeleted && table[index].key.equals(key)) {
            return table[index].value;
        }
        // Quadratic probing
        index = (hash + attempt * attempt) % capacity;
        attempt++;

        if (attempt >= capacity) break;
    }

    return null;  // Key not found
}

public void remove(K key) {
    if (key == null) return;

    int hash = getHash(key);
    int index = hash;
    int attempt = 1;

    while (table[index] != null) {
        if (!table[index].isDeleted && table[index].key.equals(key)) {
            table[index].isDeleted = true;  // Lazy deletion
            size--;
            return;
```

```
            }
            // Quadratic probing
            index = (hash + attempt * attempt) % capacity;
            attempt++;

            if (attempt >= capacity) break;
        }
    }

    @SuppressWarnings("unchecked")
    private void resize() {
        int oldCapacity = capacity;
        capacity *= 2;
        Entry<K, V>[] oldTable = table;
        table = (Entry<K, V>[]) new Entry[capacity];
        size = 0;

        // Rehash all existing entries
        for (int i = 0; i < oldCapacity; i++) {
            if (oldTable[i] != null && !oldTable[i].isDeleted) {
                put(oldTable[i].key, oldTable[i].value);
            }
        }
    }

    private int getHash(K key) {
        return Math.abs(key.hashCode() % capacity);
    }
}

// Fibonacci with Hash Table Memoization
public class FibonacciWithHashing {
    private static HashMap<Integer, Long> memo = new HashMap<>();

    public static long fibonacci(int n) {
        // Base cases
        if (n <= 1) return n;

        // Check if already computed
        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        // Compute and store result
        long result = fibonacci(n - 1) + fibonacci(n - 2);
        memo.put(n, result);

        return result;
    }

    // Test the implementation
    public static void main(String[] args) {
```

```
        System.out.println("Fibonacci(10): " + fibonacci(10));
        System.out.println("Fibonacci(45): " + fibonacci(45));  // Would be very slow without memoization

        // Print memoized values
        System.out.println("\nMemoized values:");
        memo.entrySet().stream()
            .sorted(Map.Entry.comparingByKey())
            .forEach(e -> System.out.println("F(" + e.getKey() + ") = " + e.getValue()));
    }
}
```

The implementation includes:
1. A complete hash table with quadratic probing for collision resolution
2. Lazy deletion to handle removal of elements
3. Dynamic resizing when the load factor is exceeded
4. Efficient hash function implementation
5. Fibonacci implementation using hash table for memoization

Key points to remember:
- Hash tables provide O(1) average case for insertions and lookups
- Collision resolution is crucial for performance
- Load factor affects the need for resizing
- Proper hash function selection is important for distribution
- Memoization with hash tables can significantly improve recursive algorithms

Would you like me to explain any specific aspect in more detail or provide additional examples?